

タスク管理アプリ

基本設計書

20XX 年 X 月 XX 日

株式会社〇〇

1. システム概要

1.1. 開発の概要

タスク管理アプリ（以下、本アプリ）は日々のタスクをデジタルデータとして一元管理し、進捗状況を可視化することで、業務効率の向上と抜け漏れ防止を実現するツールである。

1.2. 背景

タスクの増加に伴い、以下の問題が発生しており、業務に支障が発生している。

- **タスクの失念（抜け漏れ）：**
 - 口頭やメールで依頼されたタスクを記録し忘れる。
- **優先順位の不明確化：**
 - どのタスクから着手すべきか判断が遅れる。
- **進捗状況の不透明化：**
 - 各タスクが現在「未着手」なのか「進行中」なのかが本人以外に分からず、チーム内でのフォローが難しい。

1.3. 開発 / 実行環境

開発者個人の PC で完結して開発/動作確認ができる構成とする。

- 言語：Java 21
- Web フレームワーク：Spring 3.5
- データベース：H2 Database
- 動作環境：ローカル開発サーバー
- 対応ブラウザ：PC 版 Chrome

2. モデル設計

Lombok の `@Data` アノテーションを使用して、以下の **Task** モデルを定義する。テーブル定義は `schema.sql` に記述し、アプリケーション起動時に自動実行される。

Task モデル

フィールド名 / (論理名)	データ型	制約	選択肢
id (ID)	Long	PRIMARY KEY AUTO_INCREMENT	
title (タスク名)	String	NOT NULL VARCHAR(100)	
description (タスク説明)	String	NULL 許容	
priority (優先度)	String	NOT NULL CHAR(1)	'A': 高, 'B': 中, 'C': 低
status (ステータス)	String	NOT NULL VARCHAR(10) DEFAULT '未着手'	'未着手', '進行中', '完了'
createdAt (作成日時)	LocalDateTime	NOT NULL CURRENT_TIMESTAMP	
updatedAt (更新日時)	LocalDateTime	NOT NULL CURRENT_TIMESTAMP	

3. 画面・テンプレート設計

本アプリは index.html という単一のテンプレートを中心に構成し、詳細・登録・編集は JavaScript によるポップアップ（モーダル）表示で制御する。

3.1. タスク一覧画面（メイン画面）

- ヘッダー：アプリタイトル、新規登録ボタン（クリックでポップアップ表示）
- 表示項目：
 - タスク名：クリックすると「詳細画面」を表示
 - 優先度：A、B、C のラベルを表示
 - ステータス：プルダウン形式（未着手/進行中/完了）。変更時に即座に DB 保存。
 - 操作：編集ボタン、削除ボタン

3.2. タスク新規登録画面（ポップアップ）

- 入力項目：
 - タスク名
 - タスク説明
 - 優先度

※ステータスは入力項目に含めず、初期値として【未着手】で登録。
- 表示項目：
 - 閉じるボタン

3.3. タスク詳細表示画面（ポップアップ）

- 表示項目：
 - タスク名
 - タスク説明
 - 優先度
 - ステータス
 - 作成日時
 - 更新日時

※全項目を読み取り専用で表示。
- 表示項目：
 - 閉じるボタン

3.4. タスク編集画面（ポップアップ）

- 編集項目：
 - タスク名
 - タスク説明
 - 優先度
 - ステータス

※既存データを初期値としてセット。
- 表示項目：
 - 閉じるボタン

3.5. タスク削除画面（アラート）

- 表示項目：
 - 【タスク名】を削除してもよろしいですか？
 - はい
 - いいえ

4. コントローラー設計

TaskController.java に実装するロジックの定義。

4.1. tasks (GET)

- 全タスクを取得し、一覧画面を表示。

4.2. task (GET)

- 個別のタスクを取得し、詳細情報を表示。

4.3. createTask (GET / POST)

- (GET)空のフォーム画面を表示。
- (POST)入力データを受け取り、DB に保存してリダイレクト。

4.4. updateTaskStatus (POST)

- 一覧画面のプルダウン変更を検知し、該当タスクのステータスのみを更新。

4.5. editTask (GET / POST)

- (GET)既存データをセットしてフォームを表示。
- (POST)入力データを受け取り、DB を更新してリダイレクト。

4.6. deleteTask(POST)

- アラートを表示後、指定された ID のタスクを削除。

コメントの追加 [1]: spring 講座の命名規則に合わせて
いただければ！
<https://github.com/givery-technology/track-contents-training-app/blob/main/webapp-spring-advanced-practice/music-management-complete/src/main/java/com/example/springmusicapp/controller/AlbumController.java>

5. ルーティング設計

機能	メソッド	URL パス	関数名
タスク一覧	GET	/tasks	tasks
タスク詳細	GET	/tasks/{taskId}	task
タスク作成	GET / POST	/tasks/new	createTask
ステータス変更	POST	/tasks/{taskId}/update-status	updateTaskStatus
タスク編集	GET / POST	/tasks/{taskId}/edit	editTask
タスク削除	POST	/tasks/{taskId}/delete	deleteTask

コメントの追加 [2]: こちらも spring 講座に合わせていただければ！
<https://github.com/givery-technology/track-contents-training-app/blob/main/webapp-spring-advanced-practice/music-management-complete/src/main/java/com/example/springmusicapp/controller/AlbumController.java>

コメントの追加 [3]: こちらも spring 講座に合わせていただければ！
<https://github.com/givery-technology/track-contents-training-app/blob/main/webapp-spring-advanced-practice/music-management-complete/src/main/java/com/example/springmusicapp/controller/AlbumController.java>

コメントの追加 [4]: こちらも spring 講座に合わせていただければ！
<https://github.com/givery-technology/track-contents-training-app/blob/main/webapp-spring-advanced-practice/music-management-complete/src/main/java/com/example/springmusicapp/controller/AlbumController.java>

コメントの追加 [5]: こちらも spring 講座に合わせていただければ！
<https://github.com/givery-technology/track-contents-training-app/blob/main/webapp-spring-advanced-practice/music-management-complete/src/main/java/com/example/springmusicapp/controller/AlbumController.java>

コメントの追加 [6]: こちらも spring 講座に合わせていただければ！
<https://github.com/givery-technology/track-contents-training-app/blob/main/webapp-spring-advanced-practice/music-management-complete/src/main/java/com/example/springmusicapp/controller/AlbumController.java>

コメントの追加 [7]: こちらも spring 講座に合わせていただければ！
<https://github.com/givery-technology/track-contents-training-app/blob/main/webapp-spring-advanced-practice/music-management-complete/src/main/java/com/example/springmusicapp/controller/AlbumController.java>

6. 開発の進め方

6.1. 環境構築

- JDK 21 以上をインストール。

6.2. プロジェクト作成

- Spring Initializr でプロジェクトを生成。
- 依存関係: Lombok、Spring Web、Thymeleaf、JDBC API、MyBatis Framework、H2 Database を選択。

6.3. モデル実装

- entity パッケージに Task.java を作成し、@Data 等の Lombok アノテーションを付与。
- mapper パッケージに TaskMapper.java (@Mapper インターフェース) を作成。
- src/main/resources/ 配下に schema.sql (テーブル定義) と data.sql (初期データ) を作成。
- application.properties で H2 データベース接続設定を記述。

6.4. 一覧表示の実装

- controller パッケージに TaskController.java を作成。
- service パッケージに TaskService.java を作成し、ビジネスロジックを実装。
- Model.addAttribute() で Thymeleaf テンプレートヘデータを渡す処理を実装。

6.5. フロントエンド構築

- HTML/CSS によるレイアウトと、JS によるポップアップ表示制御の作成。

6.6. 各機能の実装

- 登録、更新、削除のロジックを順次実装。